

Desain dan Implementasi Sistem Pengamanan Komunikasi Suara Berbasis Enkripsi Frekuensi pada FPGA

Implementasi Sekuens DTMF Satu Arah
23-28 April 2026

Tujuan

1. Merancang segmentasi kunci (*key slicing*) di sisi pengirim.
2. Merancang FSM pengirim.
3. Merancang inisialisasi sinkronisasi awal (*Pre-Transmission Sync*).
4. Merancang deteksi dan ekstraksi data di sisi penerima.
5. Merancang akumulasi dan rekonstruksi kunci di sisi penerima.

Persiapan & Segmentasi Kunci (*Key Slicing*)

- Dirancang suatu mekanisme *key slicing* dengan memecah **shift_key** 24 bit menjadi potongan-potongan 3 bit.
- Tiap potongan ini kemudian dipetakan ke masing-masing nada DTMF (1 nada DTMF = 3 bit).

```
-- %% For interfacing with the DTMF Generator %%  
signal dtmf_lout : signed(15 downto 0); -- intermediate; Lout driven via MUX  
signal command : std_logic;  
signal dtmf_out : signed(15 downto 0);  
signal tone_digit : std_logic_vector(9 downto 0);  
signal shift_key_24bit : std_logic_vector(23 downto 0);  
signal segment_counter : unsigned(3 downto 0) := (others => '0');  
signal current_3bit_segment : std_logic_vector(2 downto 0);  
signal dtmf_digit_to_send : std_logic_vector(9 downto 0);
```

```
-- Phase 1: prepare 24-bit key source for segmentation.  
shift_key_24bit <= shift_key;
```

```
-- Combinational decoder with preamble: [0]='#', [1]='3', [2..9]=encoded key.  
SEGMENT_TO_DTMF_DECODER : process(segment_counter, current_3bit_segment)  
begin  
  case to_integer(segment_counter) is  
    when 0 =>  
      dtmf_digit_to_send <= "1000000001"; -- DTMF '#'  
    when 1 =>  
      dtmf_digit_to_send <= "0000000100"; -- DTMF '3'  
    when others =>  
      case current_3bit_segment is  
        when "000" => -- DTMF '3'  
          dtmf_digit_to_send <= "0000000100";  
        when "001" => -- DTMF '2'  
          dtmf_digit_to_send <= "0000000010";  
        when "010" => -- DTMF '4'  
          dtmf_digit_to_send <= "0000001000";  
        when "011" => -- DTMF '5'  
          dtmf_digit_to_send <= "0000010000";  
        when "100" => -- DTMF '7'  
          dtmf_digit_to_send <= "0001000000";  
        when "101" => -- DTMF '8'  
          dtmf_digit_to_send <= "0010000000";  
        when "110" => -- DTMF '0'  
          dtmf_digit_to_send <= "0000000000";  
        when others => -- "111" -> DTMF '*'  
          dtmf_digit_to_send <= "1000000000";  
      end case;  
    end case;  
  end process;
```

Perancangan FSM Pengirim

- Dirancang suatu FSM pengirim dengan *state-state* esensial: **IDLE**, **TRANSMIT**, **SILENCE**.
- Pada *state* **TRANSMIT**, tiap nada, baik untuk sinkronisasi maupun sebagai *payload* kunci, diatur agar ditransmisikan tepat selama 20 ms, yang ekuivalen dengan 640 sampel pada frekuensi *sampling* 32 kHz.
- FSM diatur agar selalu berpindah ke *state* **SILENCE** antartransmisi nada.
- Jeda diam ini ditujukan untuk me-reset energi pada filter di sisi penerima dan mencegah deteksi satu nada yang sama sebanyak dua kali (efek *bounce*).

```
-- Phase 2 FSM sender control
type state_type is (IDLE, TRANSMIT, SILENCE);
signal current_state : state_type := IDLE;
signal sample_counter : integer range 0 to 640 := 0;
signal start_transmission : std_logic := '0';
signal dtmf_tone_enable : std_logic := '0';
constant SAMPLES_20MS : integer := 640;
```

```
when TRANSMIT =>
    dtmf_tone_enable <= '1';
    if sample_counter = SAMPLES_20MS - 1 then
        sample_counter <= 0;
        current_state <= SILENCE;
    else
        sample_counter <= sample_counter + 1;
    end if;
```

```
-- Phase 2 sequential FSM for DTMF transmission timing
FSM_DTMF_TRANSMITTER : process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            current_state <= IDLE;
            sample_counter <= 0;
            segment_counter <= (others => '0');
            dtmf_tone_enable <= '0';
        else
            case current_state is
                when IDLE =>
                    dtmf_tone_enable <= '0';
                    sample_counter <= 0;
                    if start_transmission = '1' then
                        segment_counter <= (others => '0');
                        current_state <= TRANSMIT;
                    end if;
            end case;
        end if;
    end if;
```

Inisialisasi Sinkronisasi Awal (*Pre-Transmission Sync*)

- Diimplementasikan suatu mekanisme *Pre-Frame Synchronization* berupa pengiriman sinyal *mark* (#) dan *flag* (3) sebagai sekuen *preamble* sebelum mengirimkan 8 nada kunci.
- Transmisi *preamble* ini ditujukan agar modul *Frame Synchronization* penerima berhasil mendeteksi penanda awal dan secara otomatis memicu sinyal `mark_enable = 1` serta `goertzel_enable = 1`.

```
-- Combinational decoder with preamble: [0]='#', [1]='3', [2..9]=encoded key.
SEGMENT_TO_DTMF_DECODER : process(segment_counter, current_3bit_segment)
begin
  case to_integer(segment_counter) is
    when 0 =>
      dtmf_digit_to_send <= "100000001"; -- DTMF '#'
    when 1 =>
      dtmf_digit_to_send <= "000000100"; -- DTMF '3'
    when others =>
      case current_3bit_segment is
        when "000" => -- DTMF '3'
          dtmf_digit_to_send <= "000000100";
        when "001" => -- DTMF '2'
          dtmf_digit_to_send <= "000000010";
        when "010" => -- DTMF '4'
          dtmf_digit_to_send <= "000001000";
        when "011" => -- DTMF '5'
          dtmf_digit_to_send <= "000010000";
        when "100" => -- DTMF '7'
          dtmf_digit_to_send <= "001000000";
        when "101" => -- DTMF '8'
          dtmf_digit_to_send <= "001000000";
        when "110" => -- DTMF '0'
          dtmf_digit_to_send <= "000000000";
        when others => -- "111" -> DTMF '*'
          dtmf_digit_to_send <= "100000000";
      end case;
    end case;
  end process;
```

Deteksi & Ekstraksi Data

- Daya dihitung menggunakan algoritma Goertzel, terutama dengan modul filter digital bawaan, untuk memproses sinyal masukan, baik untuk nada pembentuk DTMF berfrekuensi tinggi maupun rendah.
- Nilai *power* tersebut dimasukkan ke blok komparator **DTMF Encoder** untuk diidentifikasi.
- Nada apa yang sedang dikirim akan dikenali.
- Nada kemudian dikonversi kembali menjadi representasi data 3-bit yang sesuai dengan tabel pemetaan pada proses *key slicing* sebelumnya.

```
-- =====  
-- Receiver Phase 4: Goertzel power bank + DTMF encoder chain  
-- =====  
GOERTZEL_RX : entity work.Goertzel_top  
generic map (  
    DATA_WIDTH => 16,  
    BLOCK_SIZE => 640  
)  
port map (  
    clk      => AUD_XCK,  
    rst      => not KEY(0),  
    in_ready => in_ready,  
    in_valid => goertzel_enable,  
    DTMF_sig => std_logic_vector(Lin),  
    out_ready => encoder_in_ready,  
    out_valid => goertzel_out_valid,  
    power_697 => power_697,  
    power_770 => power_770,  
    power_852 => power_852,  
    power_941 => power_941,  
    power_1209 => power_1209,  
    power_1336 => power_1336,  
    power_1477 => power_1477  
);
```

```
DTMF_ENCODER_RX : entity work.top_dtmfencode  
port map (  
    clk      => AUD_XCK,  
    rst      => not KEY(0),  
    in_valid => goertzel_out_valid,  
    in_ready => encoder_in_ready,  
    corr_697 => power_697,  
    corr_770 => power_770,  
    corr_852 => power_852,  
    corr_941 => power_941,  
    corr_1209 => power_1209,  
    corr_1336 => power_1336,  
    corr_1477 => power_1477,  
    out_ready => '1',  
    out_valid => out_valid,  
    sevseg    => HEX0,  
    anode     => anode,  
    encode_out => encode_out,  
    dtmf_code_4bit => dtmf_code_4bit  
);
```

```
dec_DTMF : component decision  
port map(  
    clk      => clk,  
    rst      => rst,  
    in_valid => v2v2,  
    out_ready => r2r1,  
    in_ready => r2r2,  
    out_valid => v2v3,  
    code_low  => code_low,  
    code_high => code_high,  
    anode     => anode,  
    seg_out   => sevseg,  
    dtmf_code => code_dtmf  
);
```

Akumulasi & Rekonstruksi Kunci

- *Output* 3-bit dari *DTMF Encoder* dihubungkan ke modul `shift_add:encoder`.
- Setiap kali data 3-bit yang valid masuk, susunan bit akan digeser (*shift*) ke kiri dan ditambahkan (*add*) ke dalam register.
- Proses ini diulangi tepat delapan kali sesuai jumlah segmen nada DTMF yang diterima.
- Setelah 8 siklus akumulasi selesai, akan dihasilkan *output* 24-bit utuh (`output32[23..0]`).
- Kunci utuh final yang siap didistribusikan ke subsistem dekripsi (*Descrambling*) dinyatakan oleh *output* ini.

```
SHIFT_ADD_RX : entity work.shift_add
port map (
    clk      => AUD_XCK,
    reset    => not KEY(0),
    in_valid => out_valid,
    out_ready => '1',
    in_ready => shift_add_in_ready,
    out_valid => shift_add_out_valid,
    input3   => dtmf_code_4bit,
    output32 => reconstructed_key_24bit
);
```

```
-- body --
    clk <= AUD_XCK;
    rst <= not KEY(0);
    start_transmission <= command;
    goertzel_enable <= enable;
    shift_key <= reconstructed_key_24bit;
```

```
when COMPUTE =>
    if input3(3) = '1' then
        temp_sig <= std_logic_vector(shift_left(unsigned(temp_sig), 3));
        temp_sig(2 downto 0) <= input3(2 downto 0);
        counter <= counter + 1;
    else
        temp_sig <= temp_sig;
        counter <= counter;
    end if;
state <= STORE;
```

Hasil

- Modul *Top Level* berhasil dikompilasi secara utuh.

```
# ** Note: (vsim-220) 'modelsim.ini' is used as the ini file.
# vsim -quiet -t lps -lib work work.AcakCakap_Top
# Start time: 14:18:18 on Apr 27,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt1
#
# quit -sim
# End time: 14:18:30 on Apr 27,2026, Elapsed time: 0:00:12
# Errors: 0, Warnings: 0
#
# puts ""
#
# puts {=== SUCCESS: Compile and elaboration passed for work.AcakCakap_Top ===}
# === SUCCESS: Compile and elaboration passed for work.AcakCakap_Top ===
```


Rencana

- Memverifikasi fungsionalitas keseluruhan sistem menggunakan *testbench* global.
- Melakukan sintesis kode ke dalam *file bitstream* Quartus.
- Melakukan pengujian perangkat keras secara langsung melalui *Hardware Loopback Test* dan *VoIP Test*.

Tujuan

1. Melakukan perbaikan *frame synchronization*

Informasi tentang *Frame Synchronization*

1. Spesifikasi: Frekuensi *sampling* 32kHz dan durasi 20ms
2. Satu nada DTMF = 640 sampel
3. Kombinasi *Frame* dan *Batch* -> Menghasilkan 640 sampel sesuai nada DTMF
 - a. 16 sampel/frame dan 40 frame/batch
 - b. 20 sampel/frame dan 32 frame/batch
 - c. 32 sampel/frame dan 20 frame/batch -> Sudah menjadi pilihan terbaik berdasarkan simulasi MATLAB
 - d. 40 sampel/frame dan 16 frame/batch
4. Skema alir pendeteksian sinyal *flag* atau “#” (Frekuensi 941 Hz dan Frekuensi 1477 Hz)
 - a. Jika **nilai akumulasi batch** sekarang 3 kali lebih besar daripada 32 batch sebelumnya, **count** akan bertambah 1
 - b. Jika **count** mencapai 5 kali, sinyal **flag** deteksi salah satu frekuensi menyala
 - c. Jika **semua sinyal flag frekuensi (Flag 941 Hz dan flag 1477 Hz)** menyala, deteksi “#” berhasil
5. Skema alir pendeteksian sinyal *mark* atau “3” (Frekuensi 697 Hz)
 - a. Jika **nilai akumulasi korelasi batch** sekarang lebih besar daripada nilai akumulasi korelasi pada batch sebelumnya, sinyal deteksi simbol “3” akan menyala -> **Deteksi “3”** berhasil

Fakta tentang Informasi di atas

- Poin 2 tidak bisa diperbaiki dan sudah menjadi pilihan terbaik
- Poin 4 tidak diimplementasikan secara jelas di MATLAB dan deteksi “3” terlihat dari pemilihan poin 2
- Tidak ada banyak ruang *improvement* jika memperbaiki poin 3 dan/atau poin 4
- Implementasi *fixed-point* di MATLAB ternyata menggunakan frekuensi *sampling* 16kHz, tetapi memberikan hasil yang bagus

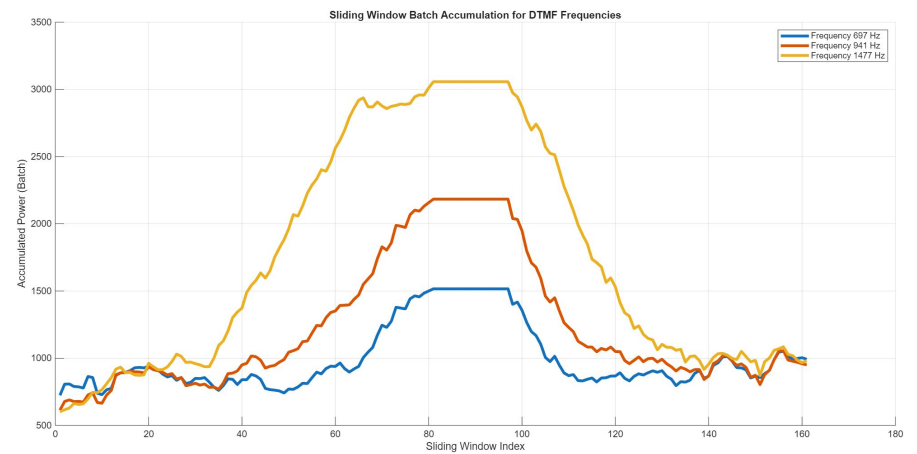
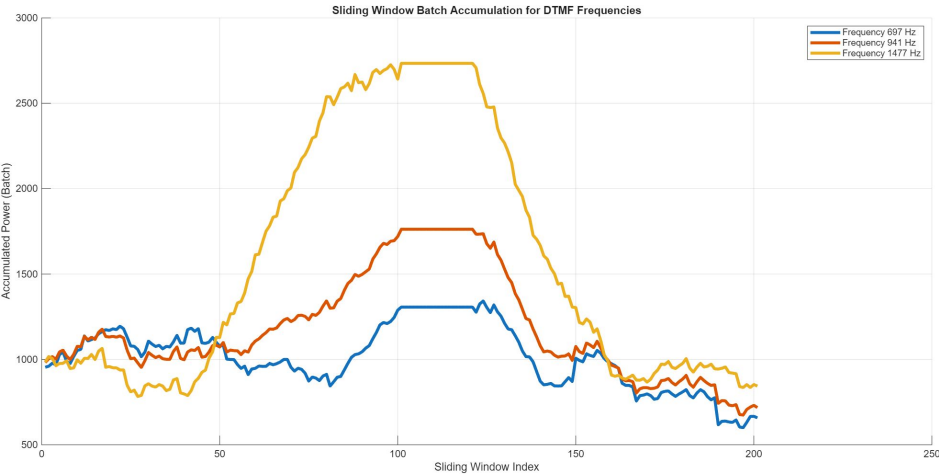
MATLAB *Frame Synchronization*

Yang sudah diimplementasikan itu **belum ada konfigurasi *fixed-point***. Dilakukan implementasi dengan referensi *fixed-point* sesuai gambar ini.

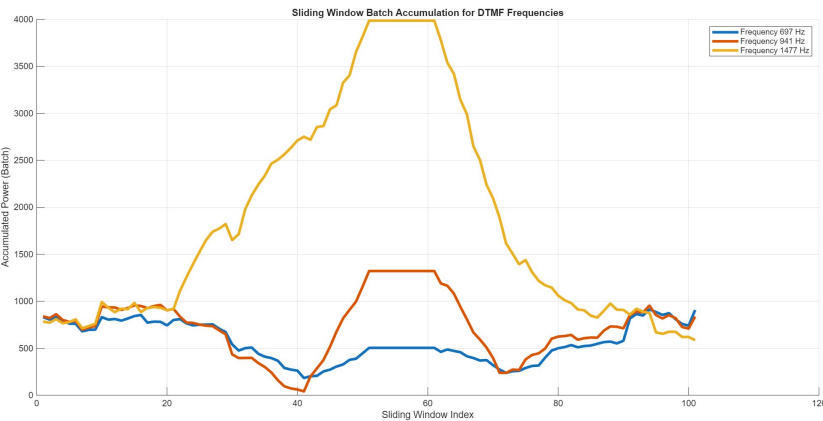
Stage	Format	Notasi
Input	2 int / 14 frac	Q2.14
Multiplier	2 / 14	Q2.14
Accumulator	6 / 10	Q6.10
Power	10 / 6	Q10.6
Batch	14 / 2	Q14.2

- Sinyal ***input*** dipertahankan di **Q2.14** dan berlaku juga untuk ***multiplier*** -> Memastikan agar tetap sama dengan sinyal ***input*** meskipun mengorbankan presisi hasil
- **Akumulator** diatur dengan format **Q6.10** karena **hasil penjumlahan *multiplier*** akan dijumlahkan **per frame**
- **Perhitungan *power*** menggunakan **Q10.6** karena perhitungannya menggunakan **operasi *squared***
- **Batch** adalah operasi untuk **menjumlahkan 20 frame** sehingga membutuhkan notasi integer bits yang lebih besar, yaitu **Q14.2**

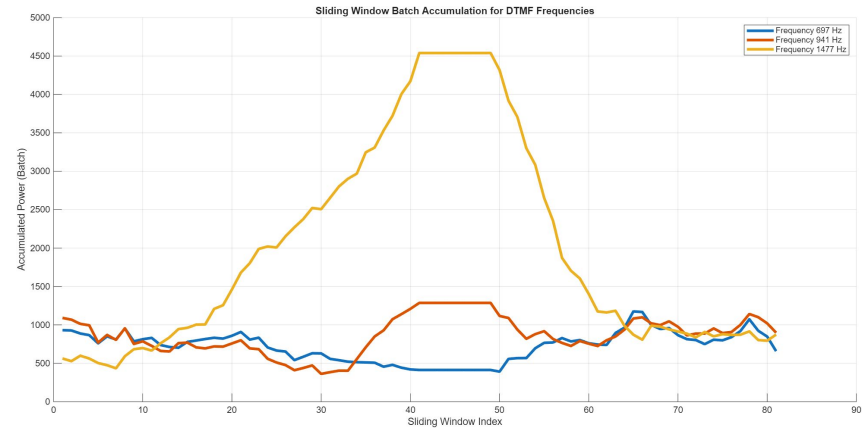
Simulasi MATLAB *Floating-Point* dari Kombinasi *Batch-Frame* (Sampling 16kHz)



16 Sampel-40 Frame



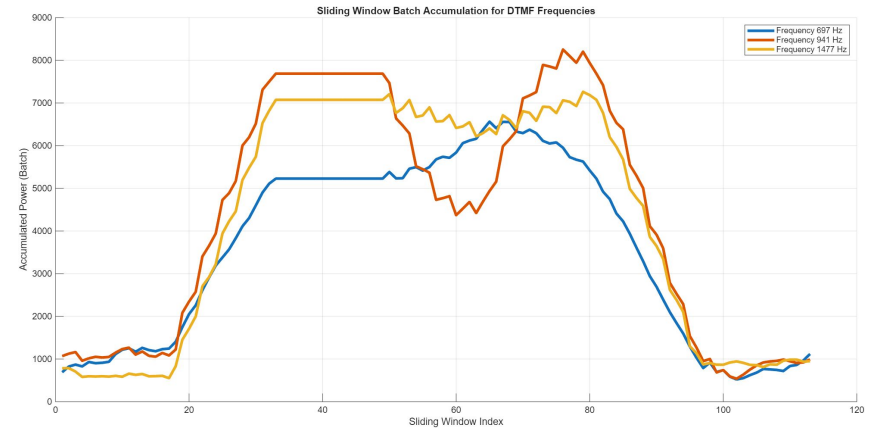
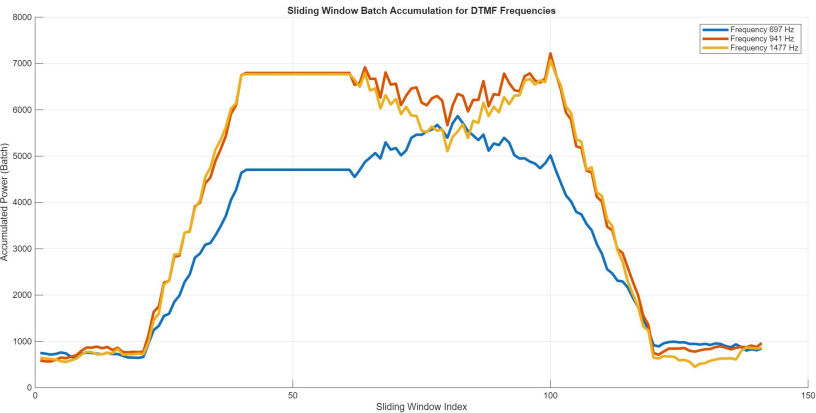
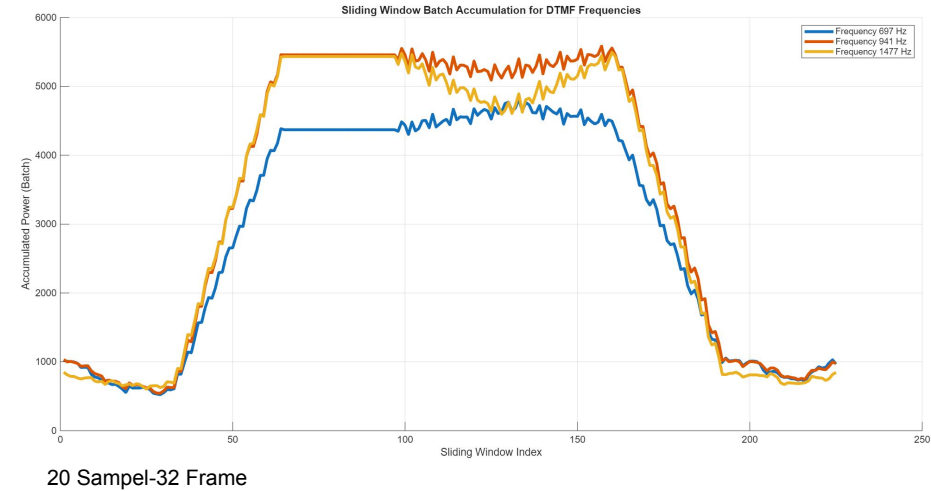
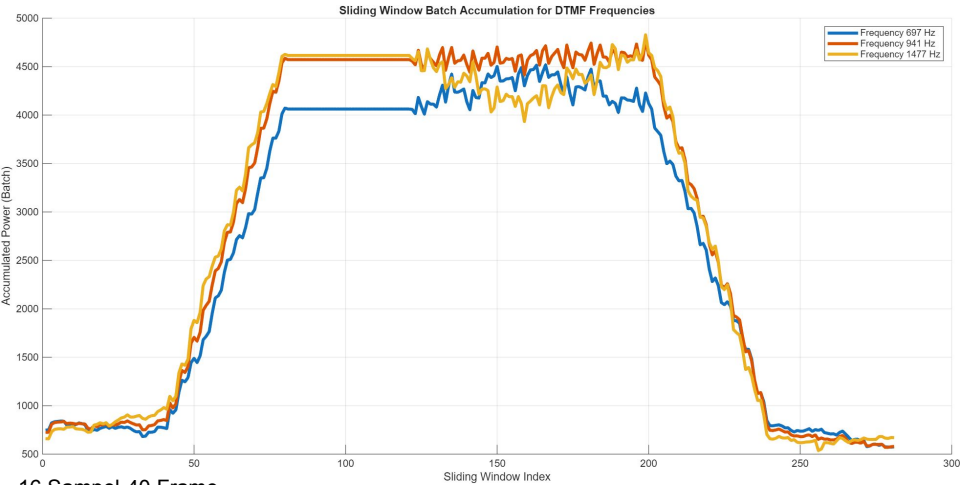
20 Sampel-32 Frame



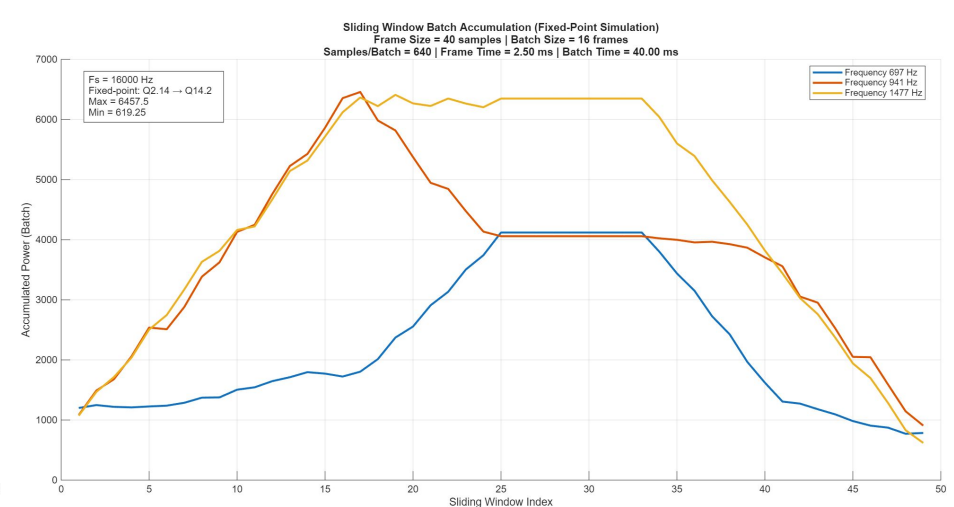
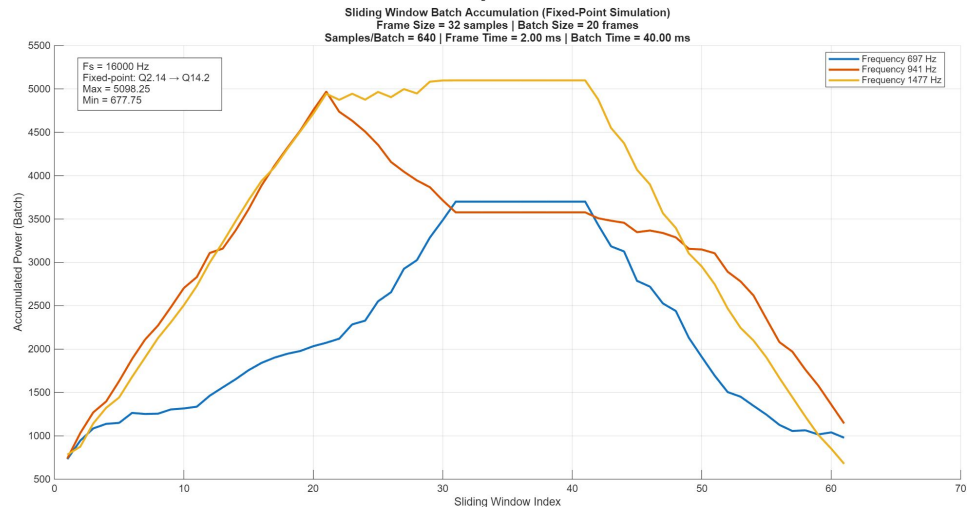
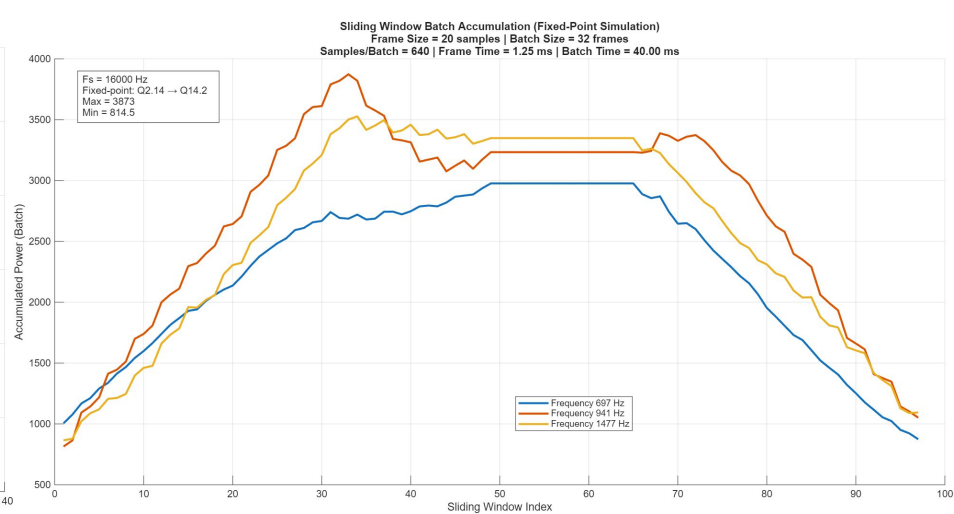
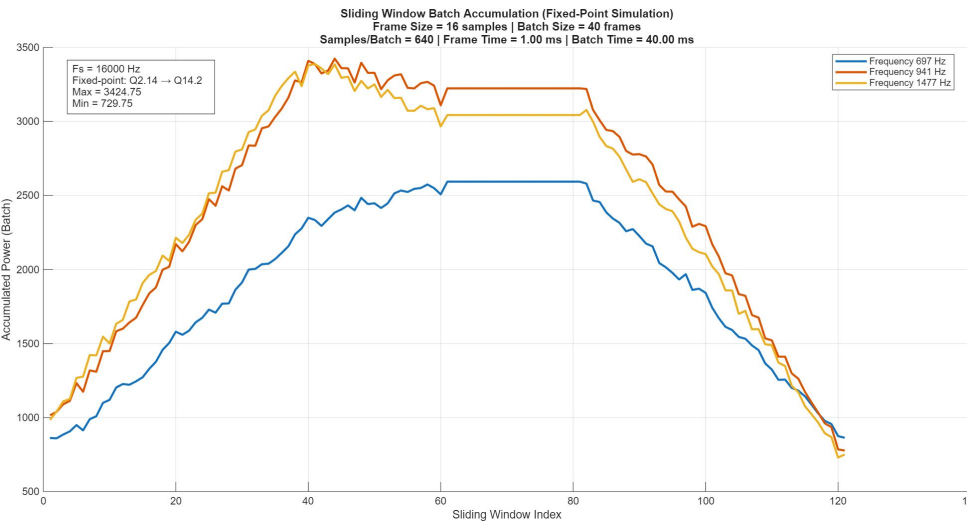
32 Sampel-20 Frame

40 Sampel-16 Frame

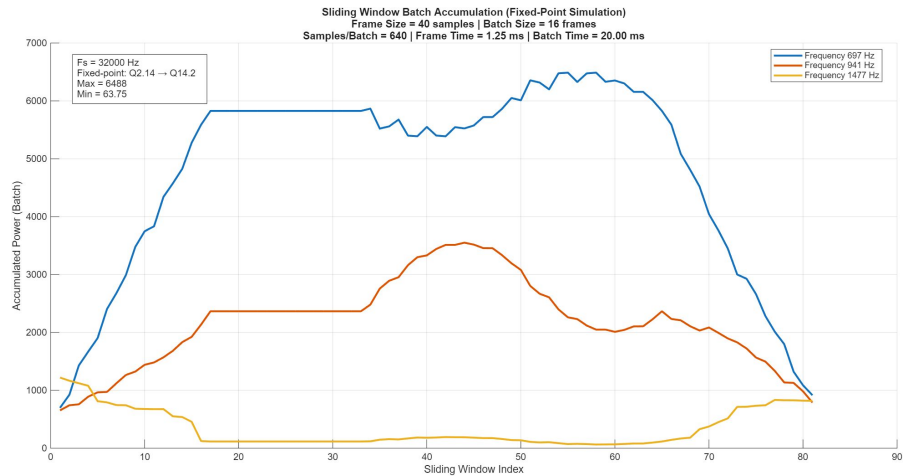
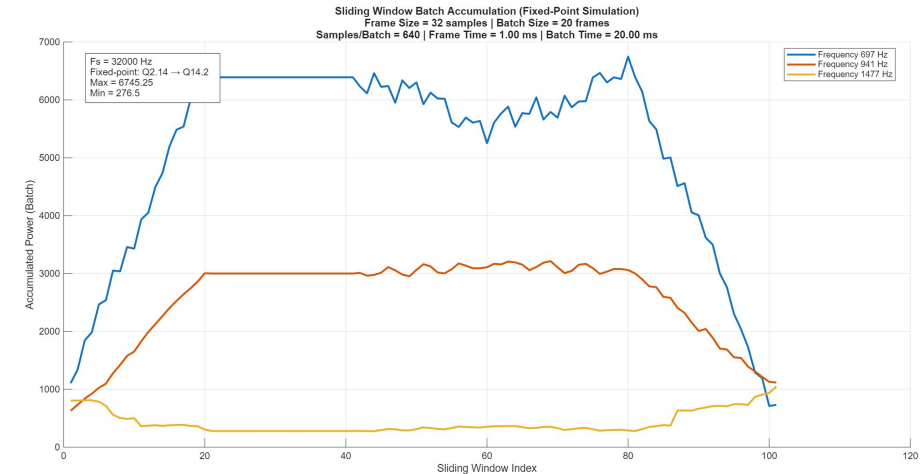
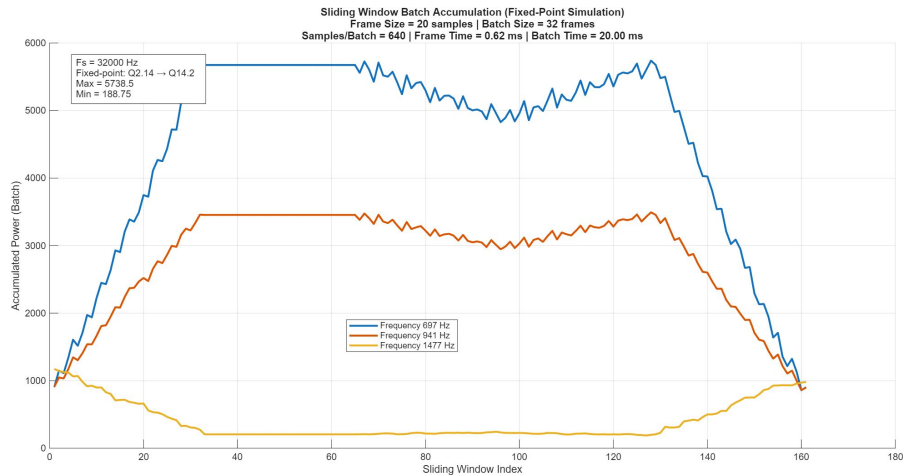
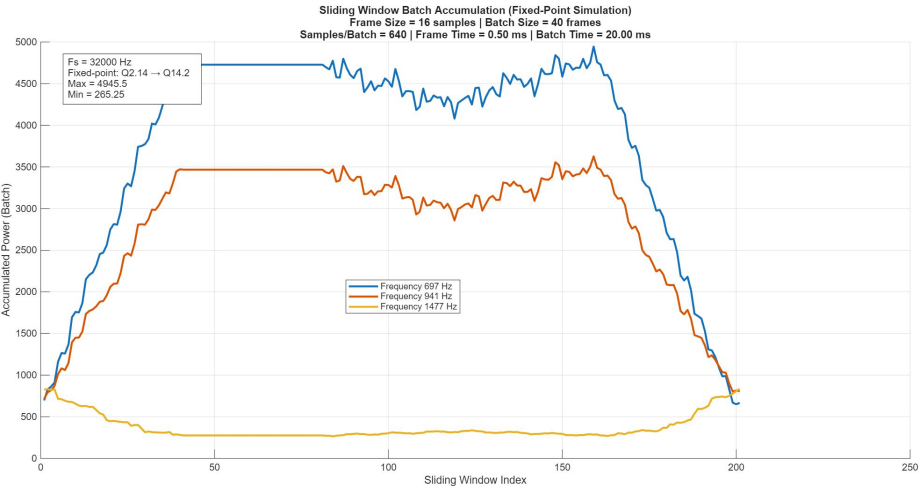
Simulasi MATLAB *Floating-Point* dari Kombinasi *Batch-Frame* (Sampling 32kHz)



Simulasi MATLAB *Fixed-Point* dari Kombinasi *Batch-Frame* (Sampling 16kHz)



Simulasi MATLAB *Fixed-Point* dari Kombinasi *Batch-Frame* (Sampling 32kHz)

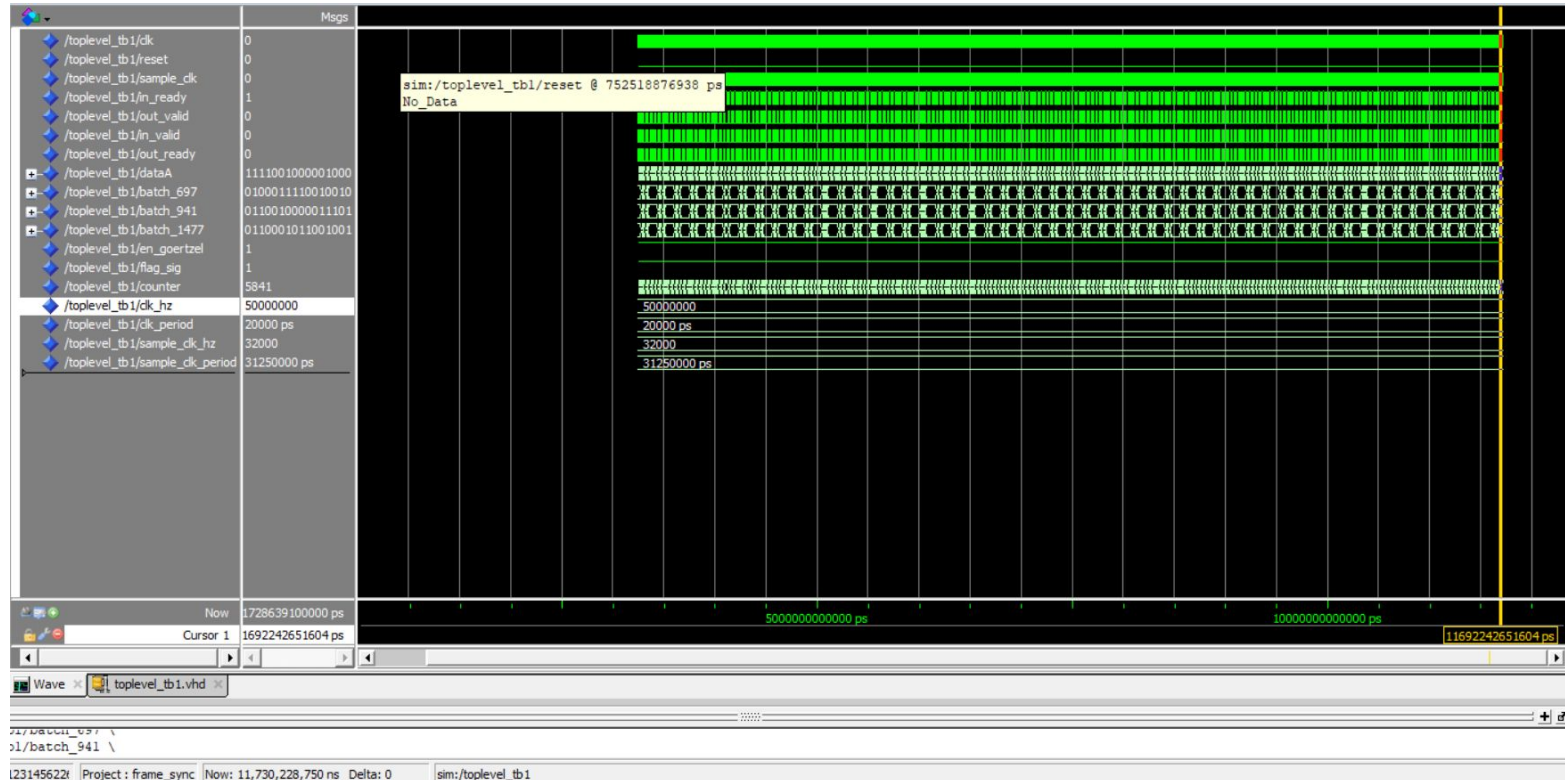


Interpretasi Simulasi MATLAB *Frame Synchronization*

- Perlu cek konsistensi hasil simulasi dari segi frekuensi *sampling*
 - Dengan *floating point*, frekuensi *sampling* 32 kHz dengan kombinasi *frame-batch* yang sudah ditentukan itu sudah benar
 - Dengan *fixed-point*, frekuensi *sampling* 16 khz dengan kombinasi *frame-batch* yang sudah ditentukan itu benar
 - Meskipun keduanya benar, frekuensi *sampling* tidak konsisten di 32 kHz

Simulasi ModelSim Frame Synchronization

- Membutuhkan waktu yang sangat lama (Masih simulasi dari pukul 19.00 sampai 22.00 lalu berhenti dan berlanjut dari pukul 23.00 sampai pukul 06.00 dan belum selesai sampai sekarang)



Rencana

- Memastikan konsistensi dari frekuensi *sampling* dan kombinasi *frame-batch* yang sesuai di MATLAB
- Memodifikasi MATLAB yang merupakan *golden reference* untuk simulasi dari implementasi *hardware* yang telah dilakukan
- Memodifikasi kode *hardware* sesuai MATLAB yang sudah dimodifikasi
- Jika *frame synchronization* sudah diimplementasikan dengan baik, gabungkan *frame synchronization* ke *top-level* dari komunikasi DTMF satu arah yang dibuat oleh rekan